

利用深度学习的城市环境数据处理和分析

1 应用概述

随着城市快速发展和工业化进程的加快，城市环境污染问题已经成为了社会关注的热点问题。针对这一问题，政府和相关部门加大了环境监管力度，要求加强城市环境信息的监控和管理。本研究旨在设计一款智能化的城市环境信息监控与分析系统，对城市空气质量和降水等重要环境因素进行监控和分析，以便及时采取相应的应对措施。涉及的技术问题包括如何精确地测量和收集环境数据，如何对各种数据进行统计、分析和建模，以及如何利用这些信息来生成相关的预测或建议。帮助相关部门更好地掌握和分析城市环境信息，进一步提高城市环境质量。

1.1 应用涉及模块

本系统主要包括以下几个模块：数据采集模块、数据存储模块、数据处理模块、数据展示模块和决策支持模块。

1.1.1 数据采集模块

主要负责采集城市环境数据，例如空气质量、降水、风速等数据。

1.1.2 数据存储模块

对采集到的数据进行存储和处理。

1.1.3 数据处理模块

对存储的数据进行一定的分析和处理，例如利用模型对空气质量进行预测、利用风速和降水数据进行城市污染范围的预测等。

1.1.4 数据展示模块

将处理后的数据以直观的方式展示给用户，例如地图、统计图表等。

1.1.5 决策支持模块

负责将分析结果提供给用户，帮助用户制定针对性的环境管理计划。

2 涉及的科学技术问题

2.1 城市环境监控数据的采集与存储

如何准确、高效地采集不同类型环境数据，如何对数据进行存储、备份和恢复等。

2.2 城市环境监控数据的预测与分析

如何利用数学模型对城市环境数据进行分析、预测，并提取有用的信息，如空气质量、降水量等。

2.3 数据展示与可视化

如何将处理后的数据以图表等直观的方式展示给用户，使用户更好地理解数据并做出决策。

2.4 城市环境监察的实时处理和应用

如何快速、准确地实现数据分析和决策支持，以便相关管理部门能够及时发现并处理扰动，并更好地应对环保事件。

2.5 分布式计算技术

如何利用分布式存储和计算技术提高系统性能和可扩展性，使系统能够处理大量的、实时产生的数据。

3 相关技术分析国内外研究现状

3.1 传感器技术

传感器技术通过对环境数据进行采集和监测，为城市环境质量评价和处理提供了依据和决策。目前，各种传感器技术应用于环境数据采集和监测，例如光学传感器、电化学传感器、气体传感器和生化传感器等。这些传感器可以对诸如 PM2.5、PM10 等颗粒物、NO₂、SO₂ 等空气质量指标、以及温度、湿度等参数进行准确和稳定的监测。



图 1 相关传感器监测指标

3.2 数据科学与机器学习技术

数据科学可以帮助我们针对环境数据进行清洗、处理和分析，从而得到更准确的结论和预测结果。机器学习技术则可以对环境数据进行更高层次的分析和探索，从而建立识别模型、预测模型和优化模型等，以便对环境问题进行控制和管理。例如，使用决策树、神经网络等机器学习算法，可以对城市环境变量之间的关系进行深入分析，而使用时间序列分析方法，可以对未来空气质量和降水情况进行更精确的预测。

如 2008 的一篇会议文章提到一种基于计算智能（CI）的方法来建模每小时空气污染预测系统，该系统可以预测空气污染物变量的浓度。通过使用隐马尔可夫模型（HMM）和模糊逻辑（HMM-fuzzy）的混合方法来对与交通量和气象变量相关的地点的每小时空气污染进行建模。将该混合模型的预测性能与基于人工神经网络的其他常用工具和使用减法聚类提取规则的其他模糊工具进行了比较。该研究表明，HMM 模糊方法能够有效地对小时空气污染预测系统进行建模。

[1]

3.3 物联网技术

本文运用了相关物联网技术，结合运用物联网传感器网络和传感器数据分析，充分发挥传感器技术和互联网的作用。结合物联网的发展趋势及其运用，发掘相关技术的可行性方案及新兴使用方案。[2]

4 系统建模

4.1 模型选择

本研究采用时序数据处理模型对问题进行解决，将当前时间序列值与过去的时间序列值进行线性回归获得预测结果。

4.2 数据预处理

根据得到的数据集，对于每一变量，使用 python 绘图包将其可视化，观察其变化情况。代码如下所示：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_excel('weather_data.xlsx',usecols="B:J") #用 pandas 读取数据
data=np.array(df) #转化为 numpy array
templ=data[:,0] #温度 1
rhum1=data[:,1] #湿度 1
temp2=data[:,2] #温度 2
rhum2=data[:,3] #湿度 2
pres=data[:,4] #气压
winds_avg=data[:,5] #平均风速
windd_avg=data[:,6] #平均风向
```

```

dewpoint=data[:7] #露点温度
Rain_avg=data[:,8] #平均降雨概率
plt.rcParams["font.sans-serif"] = ["SimHei"]
timestep=np.arange(data.shape[0])
plt.figure(figsize=(8,5))
plt.plot(timestep,Rain_avg, label=' 平均降雨概率随时间变化')
plt.xlabel("时间(从 2020/9/1 0:00:00 开始每分钟测一次)/minute")
plt.ylabel("平均降雨概率/0.1 表示 10%")
plt.legend()
plt.show()#展示绘图结果

```

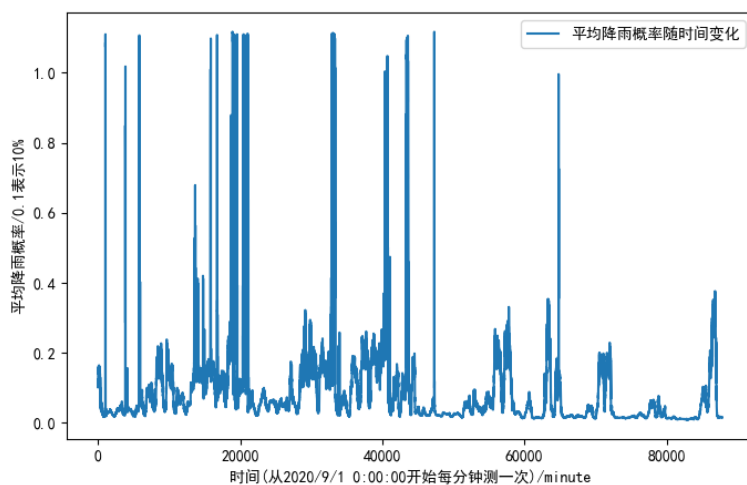


图 2 平均降雨概率随时间变化图

读入数据然后使用 MinMax 函数归一化，按照训练集和测试集 8：2 的划分比例将数据划分。

4.3 数据特征选择

记 8 个特征数据的大小为 feature_size, 1 个标签的大小为 label_size, sequence_data 的维度为 (sequence_len, feature_size+label_size)。上述处理逻辑如图所示，举例：有 8 条数据，假设 sequence_len=7



图 3 相关处理逻辑

5 算法设计

5.1 时序模型训练

设定一个时序(序列)单元，它的长度为 `sequence length`，比如第 1 分钟的数据到第 8 分钟的数据作为一个时序单元，我们将这个时序单元输入模型进行训练。具体来说，对于下图的 LSTM 模型来说 `state` 和 `h` 为两个向量，在 $t=0$ 时可以初始化为 0， $\hat{Y}$ 是一个标量。RNN cell 和 FC 中存储着计算参数。在 $t=1$ 时，我们将时序单元第 1 分钟的数据和 `state_0`、`h_0` 输入模型，产生 `state_1`、`h_1`、 $\hat{Y}_1$ ；在 $t=2$ 时，我们将时序单元第 2 分钟的数据和 `state_1`、`h_1` 输入模型，产生 `state_2`、`h_2`、 $\hat{Y}_2$ （`state` 和 `h` 保留的是当前时刻之前的时序数据信息， $\hat{Y}$ 是当前时刻预测的降雨概率）；依次类推直到将时序单元第 8 分钟的数据和 `state_7`、`h_7` 输入模型产生输出 `state_8`、`h_8`、 $\hat{Y}_8$ 。最后我们将从第 1 分钟到第 8 分钟的 $\hat{Y}$ 预测值与真实的降雨概率做 `MSE_LOSS`，然后进行梯度回传更新 RNN cell 和 FC 中存储着的计算参数。

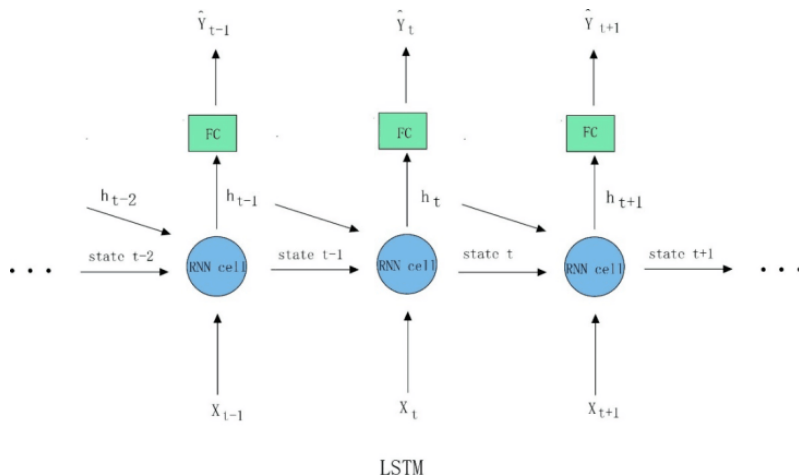


图 4 相关数据处理过程

由于将时序单元一个一个输入模型进行训练太耗费时间，所以设定一个 `batch_size`，比如说一次将 64 个时序单元输入模型进行训练，那么上述 LSTM 模型中的 `state` 和 `h` 就变成两个矩阵，存储 64 个时序单元在相应时刻之前的时序信息， $\hat{Y}$ 就变成了一个 64 维的向量。假设时序单元的长度为 8，具体来说就是在 $t=1$ 时刻，模型输入 64 个时序单元中每一个单元的第 1 个数据和 `state_0`、`h_0`，产生 64 个时序单元的 `state_1` 和 `h_1` 信息和 $\hat{Y}_1$ ，在 $t=2$ 时刻，模型输入 64 个时序单元中每一个单元的第 2 个数据和 `state_1`、`h_1`，产生 64 个时序单元的 `state_2` 和 `h_2` 信息和 $\hat{Y}_2$ ，依次类推，直到模型输入 64 个时序单元中每一个单元的第 8 个数据和 `state_7`、`h_7`，产生 64 个时序单元的 `state_8` 和 `h_8` 信息和 $\hat{Y}_8$ 。最后我们将 64 个时序单元从第 1 个数据到第 8 个数据的 $\hat{Y}$ 预测值与真实的降雨概率做 `MSE_LOSS`，然后进行梯度回传更新 RNN cell 和 FC 中存储着的计算参数。

每次我们将 `batch_size` 个时序单元输入模型，但对于指定的 `batch_size`，序列单元的个数可能不能整除 `batch_size`，比如说缺少 k 个序列单元，这时候就把 `sequence_data` 的前 `batch_size-k` 个序列单元补到 `sequence_data` 的后面。

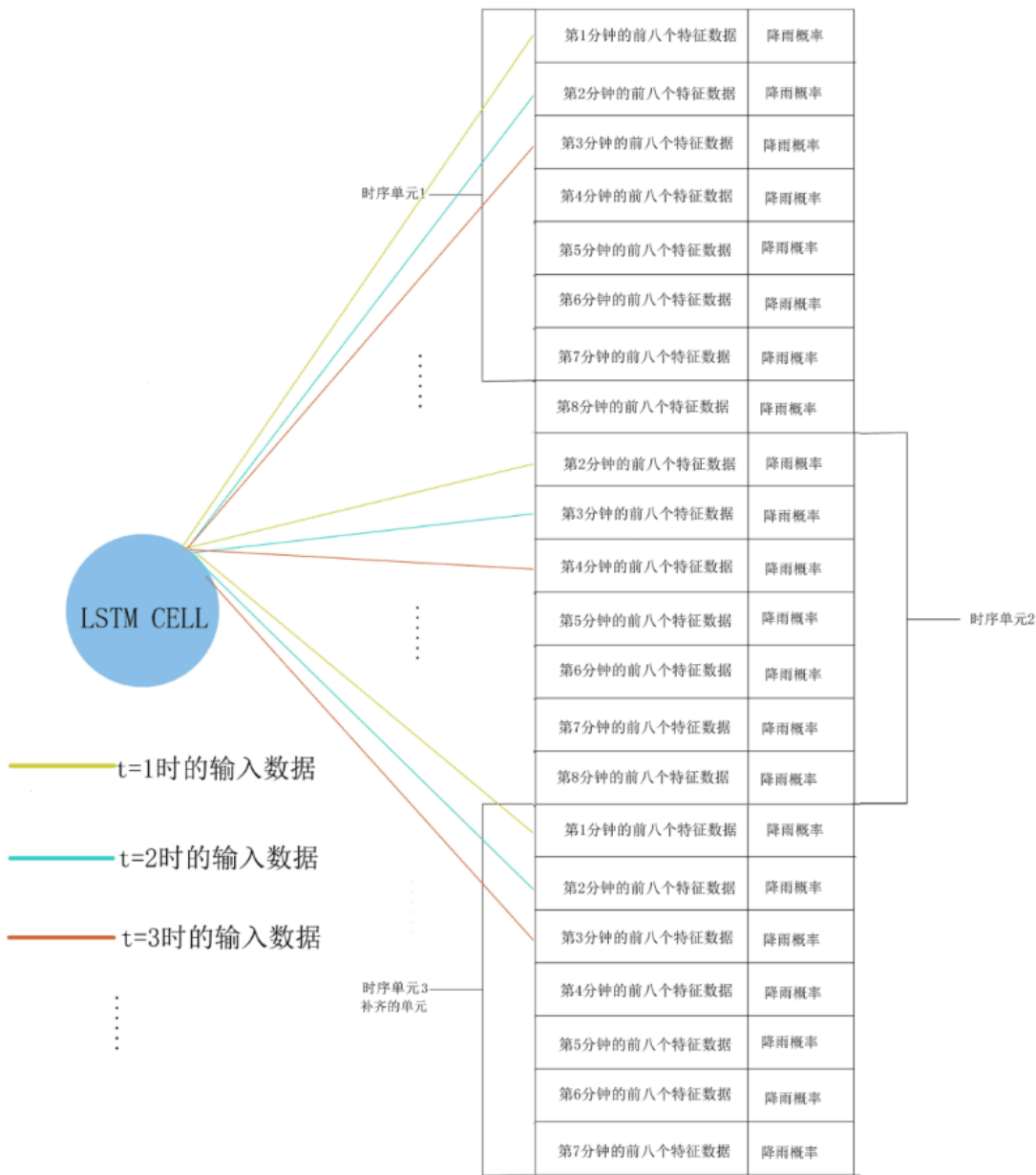


图 5 批处理的训练数据在模型中的输入

6 数据及实验结果分析

本实验使用了 Simple RNN、LSTM、GRU 模型，为了运用这些模型进行预测并比较性能，任务是：基于天气数据集的某一时刻的前 8 个特征，预测当前时刻的降雨概率。

从模型在任务上的预测准确率上来看，Simple RNN $\approx$ LSTM $\approx$ GRU。这是因为训练集只有 5000 条的天气数据，数据量太小，虽然 Simple RNN 模型的参数量少于 LSTM 和 GRU 模型的参数量，

但对于 5000 条数据来说 Simple RNN 模型的参数量已经足够，同时时序单元的长度仅为 8，LSTM 和 GRU 模型对于建模长序列数据的优势没有发挥出来。LSTM 和 GRU 在 epoch_0 处的测试损失较少的原因是 LSTM 模型的参数量比 GRU 和 Simple RNN 模型的参数量大，GRU 的参数量比 Simple RNN 模型的参数量大，这些参数被初始化为分布在 $-0.1, 0.1$ 的正态分布，这些小于 1 的参数在模型中越多，最后相乘出来的结果越小，而正好降雨概率是分布在 0-1 之间的数，且大部分分布在 0.1 附近，所以刚开始的测试损失 $LSTM < GRU < Simple\ RNN$ 。

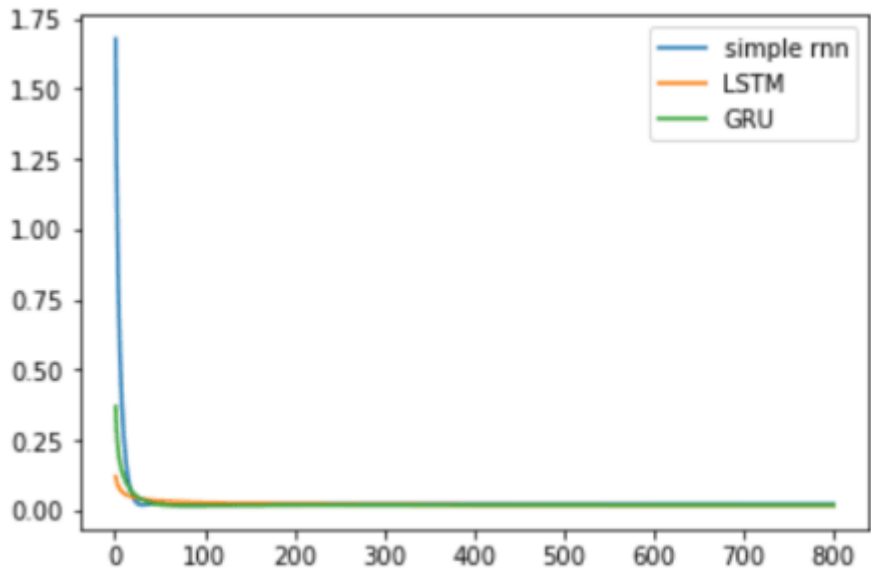


图 6 模型运行的测试结果随 epoch 的增大（横坐标表示第 k 个 epoch，纵坐标表示测试 mse_loss）

7 总结与展望

研究采用现代化的传感器技术和数据科学方法，建立了一个据处理和分析模型，以准确预测城市的空气质量和降水情况，通过实现对环境数据的实时监测和分析，从而及时发现和处理问题，并为城市环境质量的控制和管理提供有力支持。

研究未来可利用优化算法和深度学习技术进一步优化模型的效果和精度，并结合新型传感器和无线通讯技术，进行全面而深入的数据分析和处理，以获得更加准确的预测。

参考文献:

[1] M. M. Hossain, M. R. Hassan, M. Kirley, Forecasting urban air pollution using hmm-fuzzy model, in: Advances in Knowledge Discovery and Data Mining, Springer, 2008, pp. 572–581.

[2] Krishnamurthi, R.; Kumar, A.; Gopinathan, D.; Nayyar, A.; Qureshi, B. An Overview of IoT Sensor Data Processing, Fusion, and Analysis Techniques. *Sensors* **2020**, *20*, 6076. <https://doi.org/10.3390/s20216076>.

附录：

相关代码：

```
import tensorflow as tf
tf.compat.v1.disable_eager_execution()
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import copy

df=pd.read_excel('weather_data.xlsx', usecols="B:J") #从 excel 中读取第 B-J 列即 8 个特征及降雨概率
data=np.array(df) #将数据转化为 numpy 格式
transfer=MinMaxScaler() #sklearn 中的最大最小归一化包
data=transfer.fit_transform(data) #对数据执行归一化
train_size=5000 #训练集的大小
train_data = data[:train_size] #划分训练集
test_data = data[train_size:] #划分测试集
test_size=data.shape[0]-train_size #计算测试集的大小

sequence_len=8
def get_sequence(data,sequence_len):
    no=data.shape[0]-sequence_len+1 #最后一个能得到序列单元的数据位置
    sequence_data=[]
    for i in range(no): #从第一条数据开始到最后一个能得到序列单元的数据位置
        sequence_data.append(data[i:i+sequence_len]) #获取序列长度的数据组成一个序列单元
    return sequence_data
sequence_data=get_sequence(train_data,sequence_len) #上面代码获得的训练集

batch_size=64
rest=int(len(sequence_data)%batch_size) #如果序列单元总数不能整除 batch，那么从序列单元列表开头复制所需
数目的序列单元添加到列表末尾
if rest>0:
    temp=copy.deepcopy(sequence_data[0:batch_size-rest])
    for i in range(batch_size-rest):
        sequence_data.append(temp[i])
sequence_data=np.array(sequence_data) #序列单元列表转为 numpy 格式 矩阵形状为(序列单元个数，序列长度，
特征维度)

seque_batch=[] #存储 batch
batch_num=int(len(sequence_data)/batch_size) #计算批处理的个数
for i in range(batch_num):
    seque_batch.append(sequence_data[i*batch_size:i*batch_size
+batch_size])
seque_batch=np.array(seque_batch)
```

```

train_feature=sege_batch[:, :, 0:-1]
train_label=sege_batch[:, :, -1:]
feature_size=8
label_size=1

```

```

model_test_data=test_data[0:test_size,0:-1]
model_test_label=test_data[0:test_size,-1:]

```

```

import warnings
warnings.filterwarnings("ignore")
import tensorflow as tf

```

```

class LSTM_Cell(object):

```

```

    def __init__(self, train_data, train_label, num_nodes, feature_size, out_dim, batch_size):

```

```

        self.ix, self.im, self.ib = self._generate_w_b( #输入门参数

```

```

            x_weights_size=[feature_size, num_nodes],

```

```

            m_weights_size=[num_nodes, num_nodes],

```

```

            biases_size=[num_nodes])

```

```

        self.cx, self.cm, self.cb = self._generate_w_b( #记忆单元参数

```

```

            x_weights_size=[feature_size, num_nodes],

```

```

            m_weights_size=[num_nodes, num_nodes],

```

```

            biases_size=[num_nodes])

```

```

        self.fx, self.fm, self.fb = self._generate_w_b( #遗忘门参数

```

```

            x_weights_size=[feature_size, num_nodes],

```

```

            m_weights_size=[num_nodes, num_nodes],

```

```

            biases_size=[num_nodes])

```

```

        self.ox, self.om, self.ob = self._generate_w_b( #输出门参数

```

```

            x_weights_size=[feature_size, num_nodes],

```

```

            m_weights_size=[num_nodes, num_nodes],

```

```

            biases_size=[num_nodes])

```

```

        self.w = tf.Variable(tf.random.truncated_normal([num_nodes, out_dim], -0.1, 0.1)) #预测 label 用的

```

```

        self.b = tf.Variable(tf.zeros([out_dim]))

```

#Fully Connect

网络参数

```

        self.batch_size=batch_size

```

```

        self.num_nodes=num_nodes

```

```

        self.saved_output = tf.Variable(tf.zeros([batch_size, num_nodes]), trainable=False) #初始化 lstm 中的 h,

```

即 输 入

Fully Connect 的数据流

```

        self.saved_state = tf.Variable(tf.zeros([batch_size, num_nodes]), trainable=False) #初始化序列记忆

```

```

        self.train_data = train_data

```

```

        self.train_label = train_label

```

```

    def _generate_w_b(self, x_weights_size, m_weights_size, biases_size): #产生训练所需的参数 由 init 函数调

```

用

```

x_w = tf.Variable(tf.random.truncated_normal(x_weights_size, -0.1, 0.1))
m_w = tf.Variable(tf.random.truncated_normal(m_weights_size, -0.1, 0.1))
b = tf.Variable(tf.zeros(biases_size))
return x_w, m_w, b

```

```

def _run(self, input, output, state): #对于某一时刻 t, lstm 单元的计算过程
    forget_gate = tf.sigmoid(tf.matmul(input, self.fx) + tf.matmul(output, self.fm) + self.fb) #计算遗忘门
    input_gate = tf.sigmoid(tf.matmul(input, self.ix) + tf.matmul(output, self.im) + self.ib) #计算输入门
    update = tf.matmul(input, self.cx) + tf.matmul(output, self.cm) + self.cb #当前时刻 t 输入特征的信息
    state = state * forget_gate + tf.tanh(update) * input_gate #更新序列记忆并存储
    output_gate = tf.sigmoid(tf.matmul(input, self.ox) + tf.matmul(output, self.om) + self.ob) #计算输出门
    output = output_gate * tf.tanh(state) #输出信息, 即输入 Fully Connect 的数据流, 对应 lstm 中的 h
    y = tf.nn.relu(tf.matmul(output, self.w) + self.b) #生成预测 label 值
    return y, output, state

```

```

def loss_func(self): #训练时调用, 用于处理 batch_size 个时序单元的输入
    output = self.saved_output #初始化的 h
    state = self.saved_state #初始化的序列记忆
    loss = 0
    for i in range(self.train_data.shape[1]): #对于序列单元内的每一时刻
        y, output, state = self._run(self.train_data[:, i], output, state) #调用 lstm 单元计算
        #并返回更新后的 h

```

和序列记忆 state

```

        loss += tf.compat.v1.losses.mean_squared_error(self.train_label[:, i], y) #每一步的 loss 求和
    div = tf.cast(self.train_data.shape[1], tf.float32)
    self.saved_output = tf.Variable(tf.zeros([self.batch_size, self.num_nodes]), trainable=False) #初始化

```

#lstm 中的 h

```

        self.saved_state = tf.Variable(tf.zeros([self.batch_size, self.num_nodes]), trainable=False) #初始化序列

```

#记忆 state

```

        return loss / div #loss 为每一步 loss 求和的平均

```

class GRU_Cell(object):

```

def __init__(self, train_data, train_label, num_nodes, feature_size, out_dim, batch_size):
    self.rx, self.rb = self._generate_w_b( #重置门参数
        x_weights_size=[feature_size+num_nodes, num_nodes],
        biases_size=[num_nodes])
    self.zx, self.zb = self._generate_w_b( #更新门参数
        x_weights_size=[feature_size+num_nodes, num_nodes],
        biases_size=[num_nodes])
    self.hx, self.hb = self._generate_w_b( #更新序列记忆 h
        x_weights_size=[feature_size+num_nodes, num_nodes],
        biases_size=[num_nodes])

    self.w = tf.Variable(tf.random.truncated_normal([num_nodes, out_dim], -0.1, 0.1)) #预测 label 用

```

Fully Connect 网络参数

```
self.b = tf.Variable(tf.zeros([out_dim]))
self.batch_size=batch_size
self.num_nodes=num_nodes
self.saved_output = tf.Variable(tf.zeros([batch_size, num_nodes]), trainable=False) #初始化序列记忆 h

self.train_data = train_data
self.train_label = train_label
```

```
def _generate_w_b(self, x_weights_size, biases_size): #产生训练所需的参数 由 init 函数调用
    x_w = tf.Variable(tf.random.truncated_normal(x_weights_size, -0.1, 0.1))
    b = tf.Variable(tf.zeros(biases_size))
    return x_w, b
```

```
def _run(self, input, output): #对于某一时刻 t，GRU 单元的计算过程
    reset_gate = tf.sigmoid(tf.matmul(tf.concat([output,input],axis=1), self.rx) + self.rb) #计算重置门
    update_gate = tf.sigmoid(tf.matmul(tf.concat([output,input],axis=1), self.zx) + self.zb) #计算更新门
    output_ = tf.tanh(tf.matmul(tf.concat([output*reset_gate,input],axis=1), self.hx) + self.hb) #当前时刻 t
```

#输入特征的信息

```
output = output * (1-update_gate) + output_ * update_gate #更新序列记忆 h 并存储
y=tf.nn.relu(tf.matmul(output, self.w)+self.b) #生成预测 label 值
return y,output
```

```
def loss_func(self): #训练时调用，用于处理 batch_size 个时序单元的输入
    output = self.saved_output #初始化的 h
    loss=0
    for i in range(self.train_data.shape[1]): #对于序列单元内的每一时刻
        y,output = self._run(self.train_data[:,i], output) #调用 GRU 单元计算 并返回更新后的 h
        loss+=tf.compat.v1.losses.mean_squared_error(self.train_label[:,i],y) #每一步的 loss 求和
    div=tf.cast(self.train_data.shape[1],tf.float32)
    self.saved_output = tf.Variable(tf.zeros([self.batch_size, self.num_nodes]), trainable=False) #初始化
```

#序列记忆 h

```
return loss/div #loss 为每一步 loss 求和的平均
```

class Simple_RNN_Cell(object):

```
def __init__(self, train_data, train_label,num_nodes,feature_size,out_dim,batch_size):
```

```
self.w = tf.Variable(tf.random.truncated_normal([num_nodes, num_nodes], -0.1, 0.1)) #序列记忆 state 更
```

新

#

参数

```
self.u = tf.Variable(tf.random.truncated_normal([feature_size, num_nodes], -0.1, 0.1))
self.v = tf.Variable(tf.random.truncated_normal([num_nodes, out_dim], -0.1, 0.1)) #预测 label 用的
```

Connect 网络参数

```

self.b = tf.Variable(tf.zeros([out_dim]))
self.batch_size=batch_size
self.num_nodes=num_nodes
self.saved_state = tf.Variable(tf.zeros([batch_size, num_nodes]), trainable=False) #初始化序列记忆

```

```

self.train_data = train_data
self.train_label = train_label

```

```

def _run(self, input, state): #对于某一时刻 t, Simple_RNN 单元的计算过程
    new_state = tf.nn.tanh(tf.matmul(state, self.w)+tf.matmul(input, self.u)) #序列记忆 state 更新
    y=tf.nn.relu(tf.matmul(new_state, self.v)+self.b) #生成预测 label 值
    return y, new_state

```

```

def loss_func(self): #训练时调用, 用于处理 batch_size 个时序单元的输入
    state = self.saved_state #初始化的序列记忆
    loss=0
    for i in range(self.train_data.shape[1]): #对于序列单元内的每一时刻
        y, state = self._run(self.train_data[:,i], state) #调用 Simple_RNN_Cell 单元计算 并返回更新后的序

```

列

```

        #记忆 state
        loss+=tf.compat.v1.losses.mean_squared_error(self.train_label[:,i],y) #每一步的 loss 求和
    div=tf.cast(self.train_data.shape[1],tf.float32)
    self.saved_state = tf.Variable(tf.zeros([self.batch_size, self.num_nodes]), trainable=False) #初始化序列

```

#记忆

```

    return loss/div #loss 为每一步 loss 求和的平均

```

class rnn_model(object):

```

    def __init__(self, model_name, feature_size, out_dim, num_nodes, batch_size ,sequence_len ,
                  test_size,batch_num,epoch):

```

```

        self.model=None

```

```

        self.sequence_len=sequence_len

```

```

        self.batch_num=batch_num

```

```

        self.input_train_data = tf.compat.v1.placeholder(tf.float32, shape=[batch_size,sequence_len, feature_size]) #

```

一次输入一个 batch 的序列单元

```

        self.input_train_label= tf.compat.v1.placeholder(tf.float32, shape=[batch_size,sequence_len, out_dim])

```

```

        self.input_test_data = tf.compat.v1.placeholder(tf.float32, shape=[test_size, feature_size]) #测试数据

```

```

        self.input_test_label= tf.compat.v1.placeholder(tf.float32, shape=[test_size, out_dim])

```

```

        self.test_output = tf.Variable(tf.zeros([test_size, num_nodes]), trainable=False) #初始化 test 的 h

```

和

序列记忆 state

```

        self.test_state = tf.Variable(tf.zeros([test_size, num_nodes]), trainable=False)

```

```

        self.epoch=epoch

```

```

        if model_name=='LSTM':

```

```

        self.model=LSTM_Cell(self.input_train_data,self.input_train_label,num_nodes,
            feature_size,out_dim,batch_size)
elif model_name=='GRU':
    self.model=GRU_Cell(self.input_train_data,self.input_train_label,num_nodes,
        feature_size,out_dim,batch_size)
elif model_name=='RNN':
    self.model=Simple_RNN_Cell(self.input_train_data,self.input_train_label,num_nodes,
        feature_size,out_dim,batch_size)
else:
    raise ValueError('输入正确的模型名称')
self.loss = self.model.loss_func() #调用 loss_func 处理序列单元的输入并得出损失
self.train_step = tf.compat.v1.train.AdagradOptimizer(0.001).minimize(self.loss) #adam 优化器
self.y=None
self.test_loss=None
if model_name=='LSTM':
    self.y,_,_=self.model._run(self.input_test_data, self.test_output,self.test_state) #调用_run 用于

```

#测试数据的预测

```

        self.test_loss=tf.compat.v1.losses.mean_squared_error(self.input_test_label,self.y) #测试数据的 loss
elif model_name=='GRU':
    self.y,_,_=self.model._run(self.input_test_data,self.test_output) #调用_run 用于测试数据的预测
    self.test_loss=tf.compat.v1.losses.mean_squared_error(self.input_test_label,self.y) #测试数据的 loss
else:
    self.y,_,_=self.model._run(self.input_test_data,self.test_state) #调用_run 用于测试数据的预测
    self.test_loss=tf.compat.v1.losses.mean_squared_error(self.input_test_label,self.y) #测试数据的 loss
def train_and_test(self,train_feature,train_label,model_test_data,model_test_label): #模型训练和测试
    #如果使用 GPU
    #gpu_options = tf.compat.v1.GPUOptions(per_process_gpu_memory_fraction=0.333)
    #with tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(gpu_options=gpu_options)) as session:
    with tf.compat.v1.Session() as session:
        session.run(tf.compat.v1.global_variables_initializer()) #tensorflow 会话初始化
        print("Initialized....")
        sequence_len=self.sequence_len
        mean_loss = 0
        test_loss=[]
        for step in range(self.epoch): #epoch 为 80
            for j in range(self.batch_num): #依次输入一个 batch 的序列单元
                _, l= session.run([self.train_step, self.loss], feed_dict={
                    self.input_train_data: train_feature[j],
                    self.input_train_label: train_label[j]})
                mean_loss += l # 每一批数据的损失相加
            mean_loss = mean_loss / self.batch_num #批数据的平均损失
            print('Average loss at step %d: %f' % (step, mean_loss))
            t_l = self.test_loss.eval(feed_dict={self.input_test_data: model_test_data,
                self.input_test_label: model_test_label}) #输入测试数

```

据并

```
test_loss.append(t_l)
```

#得到

测试数据的损失

```
        print('test loss is: %f' % (t_l))
    return test_loss
```

out_dim=1 #模型的输出维度

num_nodes=64 #RNN 中隐藏层的大小

epoch=800 #seque_batch 全部数据输入模型训练的次数

```
rnn_model=rnn_model('GRU', feature_size, out_dim, num_nodes, batch_size ,sequence_len ,
                    test_size,batch_num,epoch)
```

```
test_loss=rnn_model.train_and_test(train_feature,train_label,model_test_data,model_test_label) #训练数据输入
```

```
test_loss=np.array(test_loss) #保存测试结果
```

```
np.save('GRU.npy',test_loss)
```